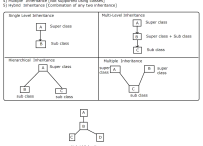
Types of Inheritance in Java :

1) Single Level Inheritance
2) Multi Level Inheritance
3) Hierarchical Inheritance
4) Multiple Inheritance [Not supported using classes]
5) Hybrid Inheritance [Combination of any two inheritance]



Hybrid Inheritance

* By default, Object is the super class for all the classes
* By default, in Java every class follows single level inheritance

//Program on Single level inheritance :

```
package com.ravi.inheritance;

class Vehicle
{
    protected int numberOfEngine = 1;

    public void startVehicle()
    {
        System.out.println("Vehicle started");
    }
}

class Car extends Vehicle
{
    protected String engineType = "Battery";

    public void carInfo()
    {
        System.out.println("Number of engine is :"+this.numberOfEngine);
        System.out.println("Engine type is :"+this.engineType);
    }
}

public class InheritanceDemo
{
    public static void main(String[] args)
    {
        Car CAR = new Car();
        CAR.startVehicle();
        CAR.carInfo();
    }
}
```

//Program on single level inheritance with encapsulation concept.

```
package com.ravi.inheritance;

class Parent
{
    private int x;
    private int y;

    public void setValue(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

    public int getX()
    {
        return this.x;
    }

    public int getY()
    {
        return this.y;
    }
}

class Child extends Parent
{
    public void showInfo()
    {
        System.out.println("x value is :"+this.getX());
        System.out.println("y value is :"+this.getY());
    }
}

public class InheritanceDemo2
{
    public static void main(String[] args)
    {
        Child child = new Child();
        child.setValue(100, 200);
        child.showInfo();
    }
}
```

Can we inherit the Constructor in the sub classes ?

* No, Constructors are not inherited in java because in sub class the name of the constructor and the parent constructor both are different.

Example :

```
package com.ravi.inheritance;

class A
{
    public A()
    {
        System.out.println("A class Constructor");
    }
}

class B extends A
{
    public B()
    {
        System.out.println("B class Constructor");
    }
}

public class ConstructorDemo
{
    public static void main(String[] args)
    {
        B b1 = new B();
        //A a1 = new A();//Valid but because a1 is a constructor but not a method

        B a1 = new A();//Valid but not recommended because there is no use of Inheritance
    }
}
```

super keyword in Java :

* It is a keyword in java which is used to access the **member/memory of super class.**
* It is always used to access the immediate super class memory OR member.

Example :

```
A
  B
    C
```

super -> It will always refer to B class member but not A class member.

* Just like this keyword, super keyword also we cannot use from static area [static context]
* super keyword we can use 3 ways in Java

a) To call the variable of super class [Variable Hiding]
b) To call the method of super class [Method Overriding]
c) To call the constructor of super class [Constructor Chaining]

1) To access super class non static field [Variable Hiding] :

Whenever super class non static field name and sub class non static field name both are same then it is called variable Hiding, Here sub class non static field hides super class non static field.

In order to access super class non static field i.e super class memory, we should use super keyword as shown in the program.

Diagram for Variable Hiding Java concept.

```
package com.ravi.inheritance;

class Employee
{
    protected double salary = 700000;

}
class Developer extends Employee
{
    protected double salary = 900000; //Variable Hiding

    public void getSalaryInfo()
    {
        System.out.println("Employee Salary is :"+super.salary);
        System.out.println("Developer Salary is :"+this.salary);
    }
}

public class VariableHiding
{
    public static void main(String[] args)
    {
        Developer developer = new Developer();
        developer.getSalaryInfo();
    }
}
```

2) To access super class non static method [Method Overriding]

If the super class non static method name and sub class non static method name both are same then Method Overriding and if we want to access for sub class then sub class method will be executed, but if we want to access for super class method from sub class method then sub class method body then we we should use super keyword as shown in the program.

```
package com.ravi.inheritance;

class Alpha
{
    public void show()
    {
        System.out.println("Alpha class show method");
    }
}
class Beta extends Alpha
{
    public void show()
    {
        super.show();
        System.out.println("Beta class show method");
    }
}

public class MethodOverridingDemo {

    public static void main(String[] args)
    {
        Beta beta = new Beta();
        beta.show();
    }
}
```